Ejercicios Grafos

Vínculos

- Clase: Algoritmos 2 Clase 6
- Definición de clases: Clases Grafos Algoritmos
- Ejercicios: Ejercicios Grafos

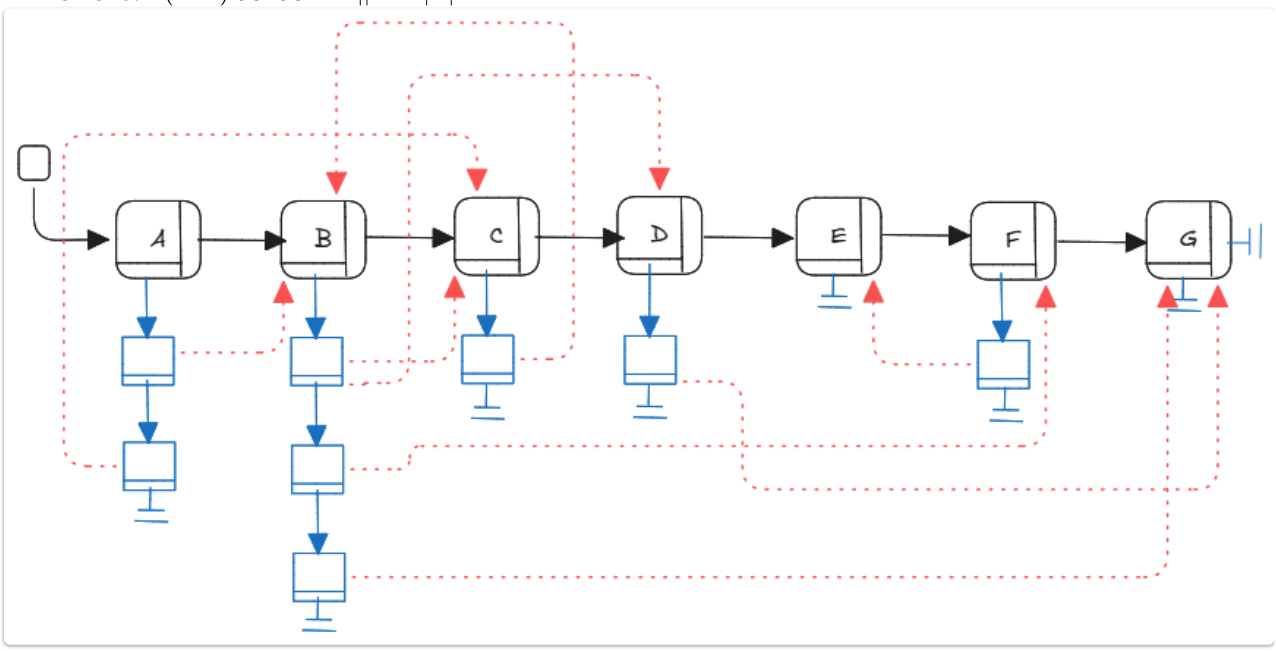
Resumen

En este documento se presentan y explican ejercicios hechos en clase y en casa del tema Estructuras Multienlazadas (Grafos), vease Algoritmos 2 Clase 6. En algunos ejercicios se utiliza inteligencia artificial para mejorar las soluciones y explicaciones de los ejercicios. A continuación la definición de las clases y la representación en memoria.

Problema	Algoritmo Recomendado	Nota Técnica
Camino más corto (no pesado)	Búsqueda en anchura	BFS
Detección de Ciclos	Búsqueda en profundidad	DFS
Conectividad	BFS o DFS	Si el recorrido visita todos los nodos.
Orden Topológico	DFS modificado	Útil para dependencias de tareas.
Laberintos	DFS	Explora hasta el fondo antes de retroceder.

> **Clases Grafos Algoritmos**
>
> ## Vínculos
>
> - Clase: Algoritmos 2 Clase 6
> - Definición de clases: Clases Grafos Algoritmos
> - Ejercicios: Ejercicios Grafos
>
> ## Representación en memoria
>
> > ### Lista de adyacencia
> >
> > Funciona con dos listas, una lista de los nodos vertices (nodos conectados linealmente) donde cada vértices tiene una lista de apuntadores a otros vértices (una lista de adyacencia).
> >
> > - Es útil en grafos no densos

- **En memoria**: $O(n\ m)$ donde $n = ||m = |A|$



Clases Nodos

Nodo Arco

```
Class NodoArco
    Attributes
        Private:
            pointer to NodoVertice<Element>: info
            pointer to NodoArco<Element>: siguiente
            float: peso
    Methods
        Public:
            proc construir(...)
            // Getters
            ...
            // Setters
            ...
endClass
```

Nodo Vértice

```
Class NodoVertice
    Attributes
        Private:
            Element: info
            pointer to NodoVertice<Element>: siguiente
            pointer to NodoArco<Element>: listaAdy
    Methods
        Public:
            proc construir(Element: info, pointer to NodoVertice<Element>: sig, pointer to
NodoArco<Element>:listaAdy)
            // Getters
            ...
            // Setters
```

```
            ...
endClass
```

Clase Grafo

> *En métodos como el DFS se utiliza un método interno privado recursivo de igual nombre con diferentes parámetros (polimorfismo).*
> *De manera similar, los métodos públicos recursivos requieren un método privado, ya sea polimórfico o no.*

```
Class Grafo
    Attributes
        Private:
            pointer to NodoVertice<Element>: g
            int: nVertices, nArcos
            bool: esDirigido
    Methods
        Public:
            proc constructor(...)
            // Getters
            ...
            // Setters
            ...

            func getVertices(): Lista<Element> // O(n)
            func getArcos(): Lista<Lista<Element>> // O(n + m)
            func getPesoArco(Element: v, w): float // O(n + arcos(v))
            func getSucesores(Element: v): Lista<Element> // O(n+ arcos(v)) Solo en dirigidos
            func getPredecesores(Element: v): Lista<Element> // O(n + m)
            func getGradoVertice(Element: v): int // O(n + arcos(v))
            func getVecinos(Element: v): Lista<Element> //O(n + arcos(v)) No dirigidos
            proc insertarVertice(Element:v) // O(1) Se inserta al principio
            proc insertarArco(Element: v, w, float: peso) // O(n)
            proc eliminarVertice(Element: v) // O(n + m)
            proc eliminarArco(Element: v, w) // O(n + arcos(v) + arcos(w))
            proc setPeso(Element: v, w, float: peso) // O(n + arcos(v) + arcos(w))
            func bfs(): Lista<Element> // O(n + m)
            func dfs(): Lista<Element> // O(n + m)
endClass
```

Mapeado de un grafo (en especial en programación)

- **Definición**: El mapeo es la técnica de asociar un `Element` (ej. un String "Caracas") con un índice entero único entre 1 y .
- **Implementación**: Generalmente se usa una **Tabla de Hash** o un Diccionario (un map de C++) dentro de la clase `Grafo`.
- **Utilidad**: Permite que el arreglo de `visitados` sea de acceso usando el índice obtenido del mapeo.

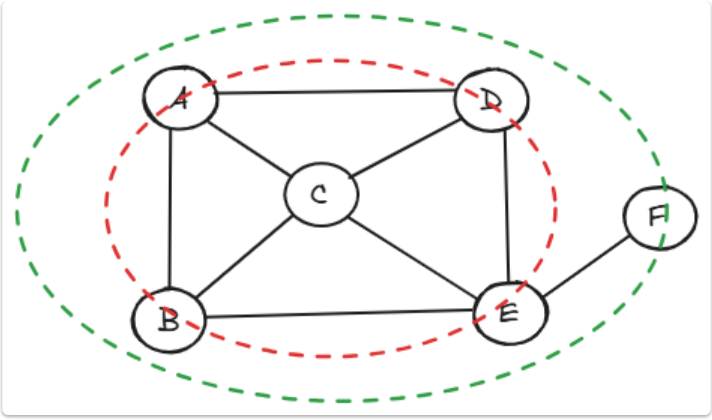
> *Se sobreentiende que en algoritmos los grafos ya están mapeados*

Implementación de Métodos Críticos

BFS (Búsqueda en anchura)

Funciona de manera parecida a un recorrido por niveles, donde se van buscando todos los vertices adyacentes al nodo, se marca como visitado y después se va iterando los vertices en busca de los demás elementos no-visitados.
Características:

1. Obtiene la componente conexa de V. (Grafo no dirigido)
2. Obtiene los nodos alcanzables. (Grafos dirigido)
3. Obtiene el camino más corto en arcos.
4. Es O(n + m)
5. Puede quedarse sin recorrer vértices no alcanzables por V.



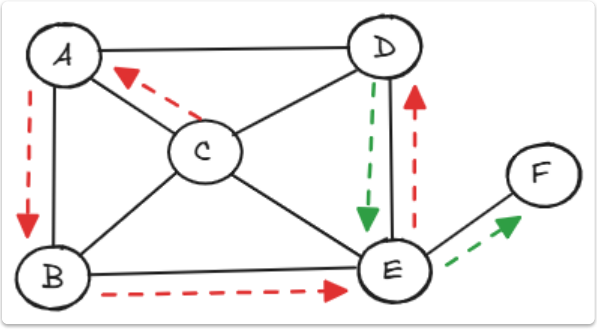

```
func BFS(Grafo<Element>: g, Element: v):
Lista<Element>
    Var
        Cola<Element>: colaAux
        array[1..N]of bool: visitados
        Lista<Elemen>: recorrido, vecinos
        int: n, i, w
    Begin
        colaAux.construir()
        recorrido.construir()
        vecinos.construir()
        n <- g.getNVertices()
        for i<-1 to N do
            visitados[i] <- false
        endfor
        colaAux.encolar(v)
        visitados[v] <- true

        while ¬colaAux.esVacia() do
            vecinos <-
g.getVecinos(colaAux.getFrente())
            while ¬vecinos.esVacia() do
                w <- vecinos.consultar(1)
                if ¬visitados[w] then
                    colaAux.encolar(w)
                    visitados[w] <- true
                endif
                vecinos.eliminar(1)
            endwhile
            recorrido.insertar(colaAux.getFrente(),
recorrido.getLong() +1)
            colaAux.desencolar()
        endwhile
        return recorrido
endfunc
```

DFS (Búsqueda en profundidad)

La búsqueda en profundidad va buscando los vértices accesibles desde el nodo inicial de manera recursiva. Cuando encuentra un nodo no-visitado lo marca como visitado y busca recursivamente los arcos con vertices no-visitados también hasta que no encuentre ninguno más.

$$stdos = \begin{array}{cccccc} A & B & C & D & E & F \\ \cancel{1} & \cancel{1} & \cancel{1} & \cancel{1} & \cancel{1} & \cancel{1} \end{array}$$

$$eorrdo = [C, A, B, E, D, F]$$



Cuando se recorre, los arcos se pueden clasificar. Esto es vital para detectar ciclos:

1. **Arcos de Árbol**: Arcos que pertenecen al bosque de expansión (cuando visitas un nodo por primera vez).
2. **Arcos de Retroceso (Back Edges)**: Arcos que van de un nodo a un ancestro en el árbol DFS. **¡Si existen, el grafo tiene ciclos!**
3. **Arcos de Avance (Forward Edges)**: Arcos que van de un nodo a un descendiente ya visitado.

4. **Arcos de Cruce (Cross Edges)**: Arcos entre ramas diferentes del árbol.

```
func DFS(Grafo<Element>: g, Element: v): Lista<Element>
    Var
        array[1..N] of bool: visitados
        Lista<Element>: recorrido
        int: i, n
    Begin
        recorrido.construir()
        n <- g.getNVertices()
        for i <- 1 to n do
            visitados[i] <- false
        endfor

        DFS(g, v, visitados, recorrido)
        return recorrido
endfunc
proc DFS(ref Grafo<Element>: g, Element: v, ref array[1..N] of bool: visitados, ref Lista<Element>: recorrido)
    Var
        Lista<Element>: vecinos
        Element: w
    Begin
        recorrido.insertar(v, recorrido.getLong() +1)
        visitados[v] <- true
        vecinos <- g.getVecinos(v)
        while ¬vecinos.esVacia() do
            w <- vecinos.consultar(1)
            if ¬visitados[w] then
                DFS(g, w, visitados, recorrido)
            endif
            vecinos.eliminar(1)
        endwhile
endproc
```

Implementación de Métodos de Clase

Eliminar Vértice

```
proc Grafo<Element>::eliminarVertice(Element: v)
    Var
        pointer to NodoVertice<Element>: ptr, target
    Begin
        *instance.eliminarArcosDeEntrada(v)
        *instance.eliminarArcosDeSalida(v)

        ptr <- instance.g
        if ptr->getInfo() = v then
            target <- ptr
            instance.g <- pre->getNext()
        else
            while ptr->getNext()->getInfo() != v do
                ptr <- ptr->getNext()
            endwhile

            target <- ptr->getNext()
            ptr->setNext(target->getNext())
        endif

        free(target)
endproc
```

```
proc Grafo<Element>::eliminarArcosDeEntrada(Element: v)
    Var
        pointer to NodoVertice<Element>: ptrV
        pointer to NodoArco<Element>: ptrA
    Begin
        ptrV <- instance.g
        while preV do
            ptrA <- ptrV->getListaAdy()
            if ptrA->getInfo->getInfo() = v then
                target <- ptrA
                ptrV.setListaAdy(target->getNext())
            else
                while ptrA->getNext()->getInfo()->getInfo() != v do
                    ptrA <- ptrA->getNext()
                endhwile
                target <- ptrA->getNext()
                ptrA->setNext(target->getNext())
            endif

            free(target)
            ptrV <- ptrV->getNext()
        endwhile
endproc
proc Grafo<Element>::eliminarArcosDeSalida(Element: v)
    Var
        pointer to NodoVertice<Element>: ptrV
        pointer to NodoArco<Element>: ptrArco, aux
    Begin
        ptrV <- instance.g
        while ptrV->getInfo() != v do
            ptrV <- ptrV->getInfo()
        endwhile

        ptrArco <- ptrV->getListaAdy()
        while ¬ptrArco do
            aux <- ptrArco
            ptrArco <- ptrArco->getNext()
            free(aux)
        endwhile
endproc
```

Guia Ejercicios

Dificultad Fácil

Representación de un grafo (Lista de Adyacencia)

Dado un número de nodos **n** y na lista de aristas (arcos), construya la representación de un grafo utilizando lista de adyacencia. El programa debe mostrar los vecinos de cada nodo.
Implemente la función: `func construirGrafo(int: n, Lista<Par<int, int>>): Lista<Lista<int>>`

> *La solución **no** debe utilizar apuntadores. O(n + m)*

```
func construirGrafo(int: n, Lista<Par<int,int>>: aristas): Lista<Lista<int>>
    Var
        Lista<Lista<int>>: result
        array[1..N] of Lista<int>: buffer // Arreglo de un tipo genérico
        Par<int, int>: arcoActual
        int: i, v, w
    Begin
```

```
        result.construir()
        // 1. Inicializamos el vector buffer (en progra se deberia crear una lista auxiliar, construirla y luego
enviarla)
        for i <- 1 to n do
            buffer[i].construir()
        endfor

        // 2. Procesamos los m vértices usando el índice del arreglo
        while ¬aristas.esVacia() do
            arcoActual <- atistas.consultar(1)
            v <- arcoActual.getPrimero()
            w <- arcoActual.getSegundo()

            buffer[v].insertar(w, 1)
            buffer[w].insertar(v, 1)

            aristas.eliminar(1)
        endwhile

        // 3. Volcamos el arreglo en la lista final con un for
        for i <- 1 to n do
            result.insertar(buffer[i], result.getLong() +1)
        endfor

        return result
endfunc
```

Verificación de existencia de vertices

Dado un grafo representado mediante matriz de adyacencia y dos nodos u y v, determine si existe una arista/arco directa entre dichos nodos.

`func existeArista(matriz: Matriz<int>, u: int, v: int): bool`

> *La solución **no** debe utilizar apuntadores. O(1)*

REQUIERE REVISION

```
func existeArista(Matriz<int>: matriz, int: u, v): bool
    Var
        int: arco
    Begin
        arco <- matriz[u][v]
        if arco = 1 then
            return true
        endif
        return false
endfunc
```

Conteo del grado de un nodo

Dado un grafo *no dirigido* representado mediante lista de adyacencia, calcule el **grado** de un nodo (cantidad de aristas/arcos incidentes)

`func gradoNodo(Grafo<int>: grafo, int: v): int`

> *La solucion no debe utilizar apuntadores. Restringido a O(grado(v))*

```
func gradoNodo(Grafo<int>: grafo, int: v): int
    Var
        Lista<int>: vecinos
    Begin
        vecinos <- grafo.getVecinos(v) // Esto es O(n + grado(v)) pero no se de que otra manera se puede hacer para
```

```
que de estrictamente O(grado(v))
        return vecinos.getLong()
```

Grafo dinámico simple

Implemente la estructura **Grafo** que permita agregar nodos, dinamicamente y conectar nodos mediante aristas.
`func Grafo<int>::agregarNodo(int: valor)`
`proc Grafo<int>::agregarArista(int: a, b)`

> *Usar Apuntadores*

```
func Grafo<int>::agregarNodo(int: valor)
    Var
        pointer to NodoVertice<int>: previo, nuevo
    Begin
        previo <- instance.verticeHead
        nuevo <- new NodoVertice<int>
        nuevo.setInfo(valor)

        nuevo.setNext(previo)
        instance.setVerticeHead(nuevo)
endfunc

proc Grafo<int>::agregarArista(int: a, b)
    Var
        pointer to NodoVertice<int> verticeActual, verticeEnvio, verticeDestino
        pointer to NodoArco<int> arcoPrevio, arcoNuevo
    Begin
        verticeActual <- instance.getVerticeHead
        while verticeActual do
            if verticeActual->getInfo() = a then
                verticeEnvio <- verticeActual
            endif
            if verticeActual->getInfo() = b then
                verticeDestino <- verticeActual
        endwhile

        if verticeEnvio ^ verticeDestino then
            arcoPrevio <- verticeEnvio->getArcoHead()

            arcoNuevo <- new(NodoArco<int>)
            arcoNuevo.setInfo(verticeDestino)
            arcoNuevo.setNext(arcoPrevio)

            verticeEnvio.setArcoHead(arcoNuevo)
        endif
endproc
```

Dificultad Media

Recorrido en Amplitud(BFS) sin apuntadores

Dado un grafo no dirigido y un vértice inicial, implemente el algoritmo BFS mostrando el orden de visita de los nodos.
`func BFS(Grafo<int>: grafo, int: inicio): Lista<int>`

```
func BFS(Grafo<int>: grafo, int: inicio): Lista<int>
    Var
        Lista<int>: vecinos, result
        Cola<int>: colaAux
        array[1..N] of bool: visitado
        int: i
```

```
    Begin
        vecinos.construir()
        for i <- 1 to grafo.getNNodos() do
            visitado.[i] <- false
        endfor

        colaAux.constuir()
        colaAux.encolar(grafo.getVerticeHead())
        while ¬colaAux.esVacia() do
            v <- colaAux.getFrente()
            vecinos <- grafo.getVecinos(v)
            visitado[v] <- true
            result.insertar(v, result.getLong() +1)

            while ¬vecinos.esVacia() do
                w <- vecinos.consultar(1)
                colaAux.encolar(w)
                vecinos.eliminar(1)
            endwhile

            colaAux.desencolar()
        endwhile
        return result
endfunc
```

Recorrido en Profundidad (DFS)

Implemente el algoritmo **DFS** en dos versiones:

- Versión **recursiva**
- Versión **iterativa** utilizando una pila

```
func dfsRecursivo(Grafo<Element>: grafo, Element: inicio)
func dfsIterativo(Grafo<Element>: grafo, Element: inicio)
```

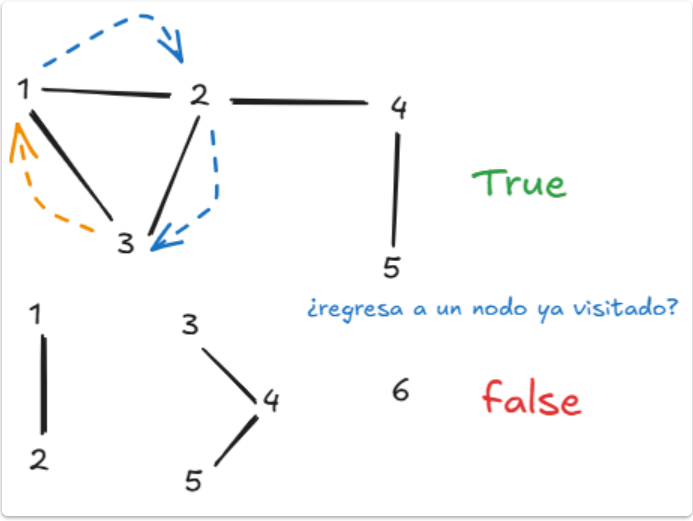
> *No usar apuntadores. O(n + m)*

```
func dfsRecursivo(Grafo<Element>: grafo, Element: inicio)
    Var
        array[1..N] of bool: visitado // estructura que guarda el mapeado del grafo (arreglo si es int, mapas si es
alguna otra)
        int: i
    Begin
        for i <- 1 to grafo.getNNodos() do
            visitados[i] <- false
        endfor

        // Inicializar salida

        // enviar dfs con salida como parametro
        dfsRecursivo(grafo, inicio, visitado)
endfunc
proc dfsRecursivo(ref Grafo<int>: grafo, Element: inicio, ref array[1..N] of bool: visitado)
    Var
        Lista<int>: vecinos
    Begin
        // Procesar Vertice
        visitado[inicio] <- true // se marca como visitado en la estructura de mapeado

        // Procesar vecinos
        vecinos <- g.getVecinos(incio)
```

```
        while ¬vecinos do
            w <- vecinos.consultar(1)
            if ¬visitados[w] then
                dfsRecursivo(grafo, inicio, visitado)
            endif
            vecinos.eliminar(1)
        endwhile
endproc

func dfsIterativo(Grafo<Element>: grafo, Element: inicio): Lista<Element>
    Var
        Pila<Element>: pilaAux
        Mapa<int, Par<Element, bool> >: visitados
        Par<Element, bool>: parAux
        Lista<Element>: vecinos, recorrido
        int: i
    Begin
        // Se llena el "vector" de visitados con falsos (se mapea)
        visitados <- mapeoVisitados()

        // Se apila el primer nodo
        parAux <- visitados[1]
        pilaAux.constuir()
        pilaAux.apliar(parAux.getFirst())

        vecinos.construir()
        recorrido.construir()

        // Se itera de manera parecida al BFS
        while ¬pilaAux.esVacia() do
            v <- pilaAux.getTope(1)
            visitados[v].setSecond(true)
            recorrido.insertar(v, recorrido.getLong() +1)
            pilaAux.desapilar()

            // Se hace como en DFS para los vecinos de v
            vecinos <- grafo.getVecinos(V)
            while ¬vecinos do
                w <- vecinos.consultar(1)
                if ¬visitados[w].getSecond() then
                    pilaAux.apliar(w)
                endif
                vecinos.eliminar(1)
            endwhile
        endwhile
endfunc

func Grafo<Element>::mapeoVisitados(): Map<int, Par<bool, Element>
    Var
        pointer to NodoVertice<Element>: actual
        Mapa<int, Par<Element, bool>>: result
        Par<Element, bool>: auxPar
        int: i
    Begin
        verAct <- instance.verticeHead
        i <- 1
        while verAct do
            auxPar.setFirst(verAct->getInfo())
            auxPar.setSecond(false)

            result.intertarUltimo(i, auxPar) // Se asume que existe el insertar como metodo de la estrutura map y
que tiene O(1)
```

```
            i <- i + 1
            verAct <- verAct->getNext()
        endwhile

        return result
endfunc
```

Detección de ciclos (Grafo Dirigido)



Se desea determinar si un grafo no dirigido tiene ciclos/circuitos en alguna parte, es decir, que en algún punto del grafo un nodo puede regresar a un nodo ya visitado antes.

Los árboles son, por definición, grafos sin ciclos.
`func tieneCiclo(Grafo<int>: g): bool`

```
func tieneCiclo(Grafo<int>: g): bool
    Var
        array[1..N] oof bool: visitado
        int: i
        bool: result
    Begin
        for i <- 1 to g.getNNodos() do
            visitados[i] <- false
        endfor

        i <- 1
        while ¬result ^ i < g.getNNodos() do
            if ¬visitados[i] then
                DFSCiclos(g, i, visitados, result, -1)
            endif
            i <- i+1
        endwhile
        return result
endfunc

proc DFSCiclo(ref Grafo<int>: g, int: v, ref array[1..N] of bool: visitados, ref bool: result, int: padre)
    Var
        Lista<int>: vecinos
    Begin
        if ¬result then
            visitados[v] <- true
            vecinos <- g.getVecinos(v)
            while ¬vecinos.esVacia() ^ ¬result do
                w <- vecinos.consultar(1)
                result <- w != padre ^ visitados[w]
```

```
                if ¬visitados[w] then
                    DFSCiclo(g, w, visitados, result, v)
                endif
                vecinos.eliminar(1)
            endwhile
        endif
endProc
```

Componentes conexas

Dado un grafo previamente mapeado encontrar la cantidad/numero de componentes conexas de un grafo. O(n+m)

Las componentes conexas son el conjunto de elementos que tienen arcos entre si. Si hay una sola componente conexas el grafo es conexo, de lo contrario se podría decir que hay varios "subgrafos" no conexos/accesibles entre si.

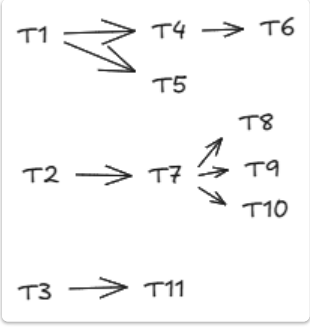
> *Se usa DFS*

```
func getNComponentesConexas(Grafo<int>: g): int
    Var
        array[1..N] of bool: visitado
        int: result, i
    Begin
        for i <- 1 to g.getNNodos() do
            visitado[i] <- false
        endfor
        result <- 0

        for i <- 1 to g.getNNodos() do
            if ¬visitado[i] then
                DFSMod(g, i, visitado)
                result <- result +1
            endif
        endfor

        return result
endfunc

proc DFSMod(ref Grafo<int>: g, int: v, ref array[1..N] of bool: visitado)
    Var
        Lista<int>: vecinos
    Begin
        vecinos <- g.getVecinos(v)
        while ¬vecinos.esVacia() do
            w <- vecinos.consultar(1)
            if ¬visitado[w] then
                visitado[w] <- true
                DFSMod(g, w, visitado)
            endif
            vecinos.eliminar(1)
        endwhile
endproc
```

Dificultad Difícil

Ordenamiento Topológico

Dado un DAG "Directed Acyclical Graph" de tareas. Devolver el orden de ejecución de estas.



Result: (T1, T2, T3, T4, T5, T7, T11, T6, T8, T9, T10)

> *Usa BFS*

```
proc calcularGradosDeEntrada(ref Grafo<int>:g, ref array[1..N] of int: grados)
    Var
        int: i, w
        Lista<int>: sucesores
    Begin
        for i <- 1 to g.getNNodos() do
            sucesores <- g.getSucesores()

            while ¬sucesores.esVacia() do
                w <- sucesores.consultar(1)
                grados[w] <- grados[w] +1
                sucesores.eliminar(1)
            endwhile
        endfor
endproc
func ordenarTareas(Grafo<int>: g): Lista<int>
    Var
        array[1..N] of int: gradosDeEntrada
        Cola<int>: colaAux
        Lista<int>: result
        int: i
    Begin
        result.construir()
        colaAux.construir()
        for i <- 1 to g.getNNodos() do
            grafosDeEntrada[i] <- 0
        endfor

        calcularGrafosDeEntrada(g, grafosDeEntrada)
        for i <- 1 to g.getNNodos() do
            if gradosDeEntrada[i] = 0 then
                colaAux.encolar(i)
            endif
        endfor

        while ¬colaAux.esVacia() do
            v <- colaAux.getFrente()
            result.insertar(v, result.getLong() +1)

            sucesores <- g.getSucesores(v)
            while ¬sucesores.esVacia() do
                w <- sucesores.consultar(1)
                colaAux.encolar(w)
                sucesores.eliminar(1)
            endwhile
            colaAux.desencolar()
```

```
        endwhile

        return result
endfunc

endfunc
```

Camino más corto con pesos (Dijkstra)

Implement el algoritmo de **Dijkstra** para encontrar las distancias mínimas desde un nodo origen todos los demás nodos del grafo con pesos positivos. Debe utilizar una cola.

`func dijkstra(Grafo<int>: grafo, int: origen): array[1..N] of float`

```
func dijkstra(Grafo<int>: grafo, int: origen): array[1..N] of float
    Var
        Lista<int>: vecinos
        Cola<int>: colaAux
        array[1..N] of bool: visitados
        array[1..N] of float: distancias
        int: i, v, w
        float: pesoArco
    Begin
        for i <- 1 to grafo.getNNodos() do
            distancias[i] <- inf
            visitados[i] <- false
        endfor

        colaAux.construir()
        colaAux.encolar(origen)
        distancias[origen] <- 0
        while ¬colaAux.esVacia() do
            extraerMinSegunDist(colaAux, distancias, v) // ya desencola el elemento
            vecinos <- grafo.getVecinos(v)
            if ¬visitados[v] then
                visitados[v] <- true

                while ¬vecinos.esVacia() do
                    w <- vecinos.consultar(1)
                    pesoArco <- grafo.getPeso(v, w)
                    if (distancias[v] + pesoArco) < distancias[w] then
                        distancias[w] <- distancias[v] + pesoArco
                        colaAux.encolar(w)
                    endif
                    vecinos.eliminar(1)
                endwhile
            endif
        endwhile
        return distancias
endfunc

proc extraerMinSegunDist(ref Cola<int>: colaAux, ref array[1..N] of float: distancias, ref int: v)
    Var
        int: actual, tamCola, i
        float: distMinima
        bool: eliminado
    Begin
        eliminado <- false
        distMinima <- inf
        tamCola <- colaAux.getLong()
        v <- colaAux.getFrente()
        for i <- 1 to tamCola do
```

```
            actual <- colaAux.getFrente()
            colaAux.desencolar()
            if distancias[actual] < distMinima then
                v <- actual
                distMinima <- distancias[actual]
            endif
            colaAux.encolar(actual)
        endfor

        for i <- 1 to tamCola do
            actual <- colaAux.getFrente()
            colaAux.desencolar()

            if (actual = v) ^ (¬eliminado) then
                eliminado <- true
            else
                colaAux.encolar(actual)
            endif
        endfor
endproc
```

Mapa de rutas entre ciudades

Modele un mapa de ciudades como un grafo no dirigido con pesos, donde cada arista/arco representa una carretera y su peso es la distancia en kilómetros.

```
func agregarCiudad(nombre)
func agregarRuta(ciudad1, ciudad2, distancia)
func ciudadesAMenosDe(ciudad, km)
func esDestinoAlcanzable(origen, destino)
```

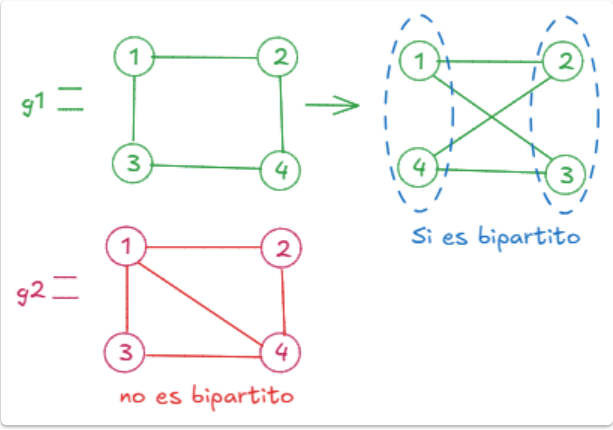
- **ciudadesAMenosDe**: retorna todas las ciudades alcanzables con una sola ruta directa de máximo *km* kilometros.
- **esDestinoAlcanzable**: retorna True si existe algún camino entre ambas ciudades.

> *Usar apuntadores*

Otros Ejercicios

Es Bipartito

Dado un grafo g conexo, mapeado y no dirigido cree la función para saber si es bipartito. Un grafo bipartito es un grafo donde se pueden extraer dos conjuntos de elementos que no esté conexos entre sí (el conjunto A no está conectado directamente con el conjunto B), no tiene un arco directo. Este problema es el mismo que el de colorear un grafo con dos colores donde los elemento contiguos no tengan el mismo color.



> *Se usa BFS*

```
func esBipartito(Grafo<int>: g): bool
    Var
        Cola<int>: colaAux
        Lista<int>: vecinos
        array[1..N] of int: color
        array[1..N] of bool: visitado
        int: i
    Begin
        for i <- 1 to g.getNNodos() do
            color[i] <- -1
            visitado[i] <- false
        endfor
        colaAux.construir()
        colaAux.encolar(1)
        color[1] <- 1

        while ¬colaAux.esVacia() do
            v <- colaAux.getFrente()
            vecinos <- g.getVecinos(v)
            visitado[v] <- true

            while ¬vecinos.esVacia() do
                w <- vecinos.consultar(1)
                if visitado[w] ^ (color[w] = color[v]) then
                    return false
                endif

                color[w] <- (color[v] +1) mod 2
                colaAux.encolar(w)
                vecinos.eliminar(1)
            endwhile
            colaAux.desencolar()
        endwhile

        return true
endfunc
```

Parcial 2-2025

Ejercicio 1: Reverse Graph

Desarrollo el método de la clase GrafoNoDirigido:
`proc GrafoDirigido<Elemento>::reverseGraph()`
Que cambie la dirección de todos los arcos(u, v) para que sean ahora arcos(v, u).

> *Usar apuntadores. O(n + m)*

```
proc GrafoDirigido<Elemento>::reverseGraph()
    Var
        pointer to NodoVertice<Element>: v, origen, destino
        pointer to NodoArco<Element>: arco, nuevoArco
        pointer to NodoVertice<Element>: copiaHead, copiaTail, actual, verticeCopia
    Begin
        // 1. se guarda una copia de los nodos con la referencia al primer arco
        v <- instance.verticeHead
        while v do
            verticeCopia <- new NodoVertice<Element>
            vertoceCopia->setInfo(v->getInfo())
```

```
            verticeCopia->setArcoHead(v->getArcoHead())
            verticeCopia->setNext(NULL)

            if verticeCopia = NULL then
                copiaHead <- verticeCopia
                copiaTail <- verticeCopia
            else
                copiaTail->setNext(verticeCopia)
                copiaTail <- verticeCopia
            endif
            v->setArcoHead(NULL)
            v <- v->getNextVertice()
        endwhile

        // 2. por cada arco origen a destino, insertar al revés destino a origen
        actual <- copiaHead
        while actual do
            v <- actual
            arco <- actual.getArcoHead()

            while arco do
                destino <- arco->getVertice() // puntero al vertice destino

                nuevoArco <- new NodoArco<Element>
                nuevoArco->setVertice(v)
                nuevoArco->setNextArco(destino->getArcoHead()) // reescribe el arco del vertice destino (ya estaba
desconectado)
                destino->setArcoHead(nuevoArco)

                arco <- arco->getNextArco()
            endwhile

            actual <- actual->getNextVertice()
        endwhile

        // Liberar backups (no se que tan necesario)
        while copiaHead do
            actual <- copiaHead
            copiaHead <- copiaHead->getNextVertice()
            free(actual)
        endwhile
endproc
```

Ejercicio 2: Camino más corto

Dado un grafo g. No dirigido, no ponderado y mapeado. Cree la función:
`func getCaminoMasCortoArcos(Grafo<int>: g, int: v, w, intermedio): Lista<int>`
Que devuelva una lista de enteros con el camino más corto en arcos de v a w pero **obligatoriamente** por el vértice *intermedio*.

> *Su solución debe ser a lo sumo O(n + m)*

```
func getCaminoMasCortoArcos(Grafo<int>: g, int: v, w, intermedio): Lista<int>
    Var
        array[1..N] of int: padres1, padres2
        Lista<int>: camino1, camino2, result
        bool: ok1, ok2
        int: vertice
    Begin
        ok1 <- BFSConPadres(g, v, intermedio, padres1)
        ok2 <- BFSConPadres(g, intermedio, w, padres2)
        result.construir()
```

```
        if ¬ok1 v ¬ok2 then
            return result
        endif

        camino1 <- reconstruirCamino(v, intermedio, padres1)
        camino2 <- reconstruirCamino(intermedio, w, padres2)

        result <- camino1
        camino2.eliminar(1)
        while ¬camino2.esVacia() do
            vertice <- camino2.consultar(1)
            result.insertar(vertice, result.getLong() +1)
            camino2.eliminar(1)
        endwhile

        return result
endfunc

func BFSConPadres(Grafo<int>: g, int: origen, destino, ref array[1..N] of int: padres): bool
    Var
        Cola<int>: colaAux
        Lista<int>: vecinos
        array[1..N] of bool: visitado
        int: i, a, b
    Begin
        for i <- 1 to g.getNNodos() do
            visitado[i] <- false
            padres[i] <- -1
        endfor

        colaAux.construir()
        colaAux.encolar(origen)
        visitado[origen] <- true

        while ¬colaAux.esVacia() do
            a <- colaAux.getFrente()
            colaAux.desencolar()

            if a = destino then
                return true
            endif

            vecinos <- g.getVecinos(a)
            while ¬vecinos.esVacia() do
                b <- vecinos.consultar(1)
                if ¬visitado[b] then
                    visitado[b] <- true
                    padres[b] <- a
                    colaAux.encolar(b)
                endif
                vecinos.eliminar(1)
            endwhile
        endwhile

        return visitado[destino] // Esto debería ser falso
endfunc

func reconstruirCamino(int: origen, destino, array[1..N] of int: padres): Lista<int>
    Var
        Pila<int>: pilaAux
        Lista<int>: camino
        int: actual
```

```
    Begin
        camino.construir()
        pilaAux.construir()

        actual <- destino
        while actual != -1 ^ actual != origen do
            pilaAux.apliar(actual)
            actual <- padres[actual]
        endwhile

        if actual = -1 then
            return camino
        endif

        pilaAux.apliar(origen)
        while ¬pilaAux.esVacia() do
            camino.insertar(pilaAux.getTope(), camino.getLong() +1)
            pilaAux.desapilar()
        endwhile

        return camino
endfunc
```